

Functional Localhost Application

Functional Localhost Application is all the product data processing components and AI templates that are put into a web application that can actually be run and used through my local application computer browser.

3 Main Pillars of Local Applications

My application uses a **Client-Server** architecture pattern. There's a front-end that users see (Frontend), a back-end that processes data (Backend), and a bridge that connects the two.

1. Backend Server

The **backend** is like the heart of the application. I use **Flask**, a lightweight yet powerful Python framework for building a web server.

- **Setting Routes (Routing):** This server acts as a traffic controller. If I'm typing in a browser, it displays the main page. If I type a chat, it displays the AI assistant interface. And if someone accesses it, it sends JSON data.
- **Security & Session (Authentication):** I added a really neat login feature. I've also added features for creating passwords, changing accounts and passwords, and checking email and password matches from the user.json file. If a user isn't logged in, the server will block access to the chat page and redirect them to the email and password entry page.

2. Frontend UI/UX

This part is what the user sees, touches, and feels. Since this is an interactive user interface (UI), I created it in three files:

- **index.html (Visual Framework and Structure):** It's used to create the basic UI appearance, including text, images, and navigation. The UI includes a sidebar, chat area, message input box, image upload button, and pop-up (modal) settings window.
- **Aesthetics (style.css):** This file will make the website look modern and elegant. I've implemented a Dark Mode system using color variables. Furthermore, the CSS will make the application responsive, meaning that when opened on a small phone screen, the sidebar can be hidden and the chat bubble automatically adjusts to the screen width. I also want to add a mobile view when opened on a phone.
- **Muscles (app.js):** JavaScript prevents the website from having to load the entire page when I send a message. Using the fetch() function, JS sends data behind the scenes. This file also controls the loading

animation, rotating the icon while waiting for the AI's response, and records the user's thumbs-up or thumbs-down feedback.

3. Connect Brain (app.py)

Even though the server (server_flask.py) handles the traffic, the heavy-lifting logic is still done in app.py. The server simply calls specific functions from app.py and then retrieves the results to return to the user.

The Overall Workflow (How Everything Connects)

The process happens in seconds when I use the application:

- **Typing and Sending:** When on localhost, I type a message and click the send button, the app.js file captures the message and displays a loading animation.
- **Journey to Server:** app.js wraps a message and sends it via the internet (locally) to server_flask.py.
- **Building AI:** server_flask.py receives the packet and brings it to app.py.
- **Think and Respond:** app.py performs context pruning, prepares a prompt, and sends it to OpenAI's API key. After OpenAI responds, the result is returned to server_flask.py.
- **Display on Screen:** server_flask.py sends the answer back to app.js, and instantly the AI answer appears on the browser screen